

UNIVERSIDAD DEL VALLE DE GUATEMALA

Reporte de entrega parcial

Servidor MCP local para gestión energética industrial

`plant-energy-mcp`

Campo	Información
Curso	CC3067 Redes
Proyecto	Proyecto 1 - Uso de un protocolo existente
Estudiante	Diego Ramírez
Fecha	20 de agosto de 2026
Alcance	Entrega parcial: servidor MCP local e incisos 8 y 10

Implementación manual de MCP sobre JSON-RPC 2.0, sin FastMCP ni SDK de MCP

1. Alcance del reporte

Este documento cubre los incisos 8 y 10 del reporte solicitados para la entrega parcial. Describe exclusivamente el servidor MCP local desarrollado y presenta conclusiones sobre esta etapa. El inciso 9, relacionado con capturas de Wireshark y análisis por capas, se reserva para la segunda entrega, cuando el servidor remoto esté disponible.

1.1 Caso de uso industrial

El caso de uso seleccionado es la gestión del consumo energético en una planta industrial. El servidor permite convertir lecturas acumuladas de medidores en información operativa: inventario de equipos, consumo por período, comparación contra umbrales y reportes resumidos. Los datos incluidos son ficticios y deterministas.

- Servidor: plant-energy-mcp.
- Lenguaje: Python 3.10 o superior, únicamente con biblioteca estándar.
- Transporte: proceso local por stdio, con un mensaje JSON por línea.
- Versión objetivo de MCP: 2025-11-25.

2. Inciso 8 - Especificación del servidor MCP

2.1 Arquitectura

La implementación separa responsabilidades para evitar que el transporte, el protocolo y las reglas del negocio queden acoplados. Esta organización facilita las pruebas y permite reemplazar posteriormente el transporte sin reescribir la lógica energética.

Componente	Responsabilidad
server.py	Lee stdin, escribe JSON-RPC en stdout y envía mensajes humanos únicamente a stderr.
protocol.py	Analiza JSON-RPC 2.0, controla el ciclo MCP, despacha métodos y normaliza errores.
tools.py	Publica las herramientas, sus inputSchema y valida el subconjunto necesario de JSON Schema.
service.py	Contiene datos ficticios, estado en memoria y reglas de cálculo energético.
demo.py	Inicia el servidor como subprocesso y demuestra una sesión MCP completa.

2.2 Transporte y ciclo de vida

El servidor no expone endpoints HTTP. En esta entrega, el endpoint lógico es el proceso local y los puntos de interacción son los métodos JSON-RPC recibidos por stdin. Cada solicitud y respuesta ocupa una sola línea.

Orden	Mensaje	Respuesta esperada
1	initialize	Negocia 2025-11-25 y devuelve capacidades e identidad del servidor.
2	notifications/initialized	Notificación del cliente; no genera respuesta.
3	ping	Devuelve un resultado vacío para comprobar disponibilidad.
4	tools/list	Devuelve cinco herramientas con nombre, descripción e inputSchema.
5	tools/call	Valida y ejecuta la herramienta solicitada.

2.3 Métodos JSON-RPC

Método	Parámetros principales	Resultado
initialize	protocolVersion, capabilities, clientInfo	Versión, capacidades, serverInfo e instrucciones.
notifications/initialized	Objeto vacío	Sin respuesta por ser una notificación.
ping	Objeto vacío	Objeto result vacío.
tools/list	Objeto vacío	Catálogo de herramientas y esquemas.
tools/call	name y arguments	content, structuredContent e isError.

2.4 Herramientas

list_equipment

Lista los equipos registrados y permite filtrar el catálogo por área.

Parámetro	Tipo	Obligatorio	Regla
area	string	No	Valores permitidos: Forming o Utilities.

```
{"name":"list_equipment","arguments":{"area":"Utilities"}}
```

record_energy_reading

Registra una lectura acumulada en la memoria de la sesión actual.

Parámetro	Tipo	Obligatorio	Regla
equipment_id	string	Sí	Debe identificar un equipo del catálogo.
timestamp	date-time	Sí	ISO 8601 con zona horaria; posterior a la última lectura.
energy_kwh	number	Sí	Entre 0 y 1,000,000,000; el acumulado no puede disminuir.

```
{"name":"record_energy_reading","arguments":{"equipment_id":"press-01","timestamp":"2026-08-20T13:00:00Z","energy_kwh":12770}}
```

calculate_consumption

Calcula el consumo y el promedio por hora entre dos lecturas acumuladas existentes.

Parámetro	Tipo	Obligatorio	Regla
equipment_id	string	Sí	Debe identificar un equipo del catálogo.
start_timestamp	date-time	Sí	Debe coincidir con una lectura registrada.
end_timestamp	date-time	Sí	Debe coincidir con una lectura posterior.

```
{"name":"calculate_consumption","arguments":{"equipment_id":"press-01","start_timestamp":"2026-08-20T08:00:00Z","end_timestamp":"2026-08-20T12:00:00Z"}}
```

detect_usage_alerts

Recibe equipment_id, start_timestamp y end_timestamp con las mismas reglas anteriores. Compara el consumo promedio con el umbral del equipo y devuelve alert y status.

get_energy_report

Resume el consumo desde la primera hasta la última lectura por equipo. Acepta area como filtro opcional con los valores Forming o Utilities.

```
{"name": "get_energy_report", "arguments": {}}
```

2.5 Respuesta de herramienta

Una ejecución correcta devuelve contenido de texto y el mismo resultado en forma estructurada.

```
{"jsonrpc": "2.0", "id": 3, "result": {"content": [{"type": "text", "text": "..."}]}, "structuredContent": {"consumption_kwh": 220.0, "average_kwh_per_hour": 55.0}, "isError": false}}
```

2.6 Errores

Código	Nombre	Situación
-32700	Parse error	La línea recibida no contiene JSON válido.
-32600	Invalid Request	La estructura JSON-RPC o el identificador son inválidos.
-32601	Method not found	El método solicitado no existe.
-32602	Invalid params	Faltan parámetros o no cumplen tipo, rango o regla de negocio.
-32603	Internal error	Fallo interno controlado sin revelar stack trace ni rutas privadas.
-32002	Server not initialized	Se intenta utilizar el servidor antes del handshake.

3. Instalación, ejecución y evidencia

3.1 Instalación

El proyecto no necesita dependencias externas. Desde una terminal limpia:

```
git clone https://github.com/Alejoram2505/redes.git
cd redes
python -m venv .venv
# Active el entorno virtual según su sistema
```

3.2 Comandos

Acción	Comando
Iniciar servidor	<code>python -m plant_energy_mcp</code>
Ejecutar demostración	<code>python demo.py</code>
Ejecutar pruebas	<code>python -m unittest discover -v</code>

3.3 Escenario demostrable

- El harness inicia plant-energy-mcp como subprocesso.
- Ejecuta initialize y luego envía notifications/initialized sin esperar respuesta.
- Descubre las cinco herramientas mediante tools/list.
- Calcula 220.0 kWh y 55.0 kWh/h para press-01 entre las 08:00 y las 12:00.
- Evalúa el período y obtiene status normal frente a un umbral de 62.0 kWh/h.
- Cierra stdin y verifica que el servidor termine con código 0.

3.4 Pruebas verificadas

La suite contiene cinco pruebas de integración. Estas inician el servidor real por stdio y verifican el handshake, el catálogo, una ejecución positiva por herramienta, los errores principales, stdout exclusivamente JSON y el cierre limpio. En la revisión del 20 de agosto de 2026 se ejecutaron cinco pruebas y todas finalizaron correctamente.

4. Inciso 10 - Conclusiones y comentario

4.1 Conclusiones

- La implementación manual permitió comprobar que MCP depende de una máquina de estados sencilla sobre JSON-RPC: primero se negocia la versión, luego se confirma la inicialización y finalmente se descubren o ejecutan herramientas.
- Separar stdout y stderr es esencial en un transporte stdio. Un banner o log humano en stdout puede romper por completo la comunicación aunque la lógica del servidor sea correcta.
- Los esquemas publicados no sustituyen la validación del servidor. Las entradas se validan nuevamente antes de ejecutar reglas de negocio para producir errores seguros y deterministas.
- La división entre transporte, protocolo, herramientas y servicio hace que el código sea comprobable y prepara una futura adaptación a un transporte remoto sin mezclar responsabilidades.
- El caso energético demuestra valor industrial sin usar datos reales: transforma lecturas acumuladas en consumo, alertas y resúmenes que un anfitrión podría presentar al usuario.

4.2 Comentario y lecciones aprendidas

La principal dificultad fue tratar el flujo como un protocolo y no como una colección de funciones. Fue necesario preservar el id de cada solicitud, no responder a notificaciones y mantener stdout libre de texto adicional. La prueba mediante subprocesos resultó más representativa que invocar directamente las funciones, porque ejercita el transporte real y detecta problemas de cierre o contaminación del flujo.

Como mejora futura, el servidor local deberá integrarse con el chatbot anfitrión. Para completar la Primera parte del proyecto todavía se requieren la conexión con un LLM, contexto conversacional por sesión, log de interacciones MCP y uso de los servidores oficiales Filesystem y Git. Estos puntos no forman parte del servidor local ya publicado, pero sí del alcance completo descrito en el documento del proyecto.

5. Limitaciones y trabajo posterior

- El estado vive en memoria y se reinicia al terminar el proceso.
- Los umbrales y lecturas son demostrativos y no constituyen recomendaciones de ingeniería.
- No existe autenticación porque el transporte actual es un subproceso local por stdio.
- La integración con el chatbot y los servidores oficiales Filesystem y Git está pendiente.
- El servidor remoto, Wireshark y el análisis por capas se reservan para la segunda entrega.

Referencias

Universidad del Valle de Guatemala. Proyecto 1 - Uso de un protocolo existente. CC3067 Redes.

Model Context Protocol. Specification 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25>

JSON-RPC Working Group. JSON-RPC 2.0 Specification. <https://www.jsonrpc.org/specification>